Analyzing the Security Properties of the Model Context Protocol (MCP) for Satellite Communications

Sebastian Altruck-Carlos, Calvin Beechner, Jesus Castro-Garcia, Noe Rosales, Jose Varela

Faculty Advisor: Mingming Chen
Project Mentor: Ed Zieglar

Abstract

This midterm report documents the progress made by KSU Team 3 during the first four weeks of the capstone project titled Analyzing the Security Properties of the Model Context Protocol (MCP) for Satellite Operations. The team has successfully transitioned from background research into active implementation and security analysis, establishing a working prototype that integrates Claude Desktop with NASA's Core Flight System (cFS) through an MCP server [4]. Key accomplishments include: setting up the MCP server infrastructure, connecting Claude Desktop to a live cFS simulation via UDP command packets, implementing new cFS application commands (including attitude control), launching the cFS Ground System GUI, and conducting in-depth research into MCP security vulnerabilities with a particular focus on prompt injection in space systems. The team is on track with the proposed project timeline and is currently working on the final presentation and report.

Keywords: Model Context Protocol (MCP); Satellite; NASA core Flight System (cFS); Prompt Injection, Satellite Software, Artificial Intelligence, Adversarial Attacks, AI Security

Contents

1 Introduction **4**

2 Overview **4**
2.1 Background . . . 4
2.1.1 MCP Architecture . . . 4
2.1.2 Satellite Communications . . . 5
2.2 Motivation . . . 5

3 Key Technologies **5**
3.1 NASA core Flight System (cFS) . . . 5
3.1.1 cFS Sample Application Extensions . . . 6
3.1.2 cFS Ground System GUI . . . 6
3.2 FastMCP . . . 6
3.3 Claude Code Desktop . . . 6
3.4 Claude Sonnet 4.6 . . . 6
3.5 Railway . . . 6
3.6 Auth0 . . . 7

4 System **7**
4.1 Architecture . . . 7
4.1.1 cFS Overview . . . 8
4.1.2 Local MCP-cFS . . . 8
4.1.3 Authentication Overview . . . 8
4.1.4 Authenticated MCP-cFS . . . 8
4.2 Functionality . . . 9
4.2.1 MCP Tools . . . 9
4.2.2 Authenticating to MCP Server . . . 10
4.3 Authentication Challenges . . . 10

5 Analysis of Vulnerabilities (Prompt Injection) **10**
5.1 Overview . . . 10
5.2 Threat Model . . . 10
5.2.1 Threat Model Explained . . . 10
5.3 Prompt Injection With cFS . . . 11
5.3.1 cFS Analysis . . . 11
5.3.2 The User . . . 12
5.3.3 AI Client . . . 12
5.3.4 MCP . . . 12
5.3.5 cFS . . . 13
5.3.6 Trust Boundaries . . . 14
5.3.7 Prompt Injection Conclusion . . . 14
5.3.8 Additional Example Prompt Injection Scenario Pre-Implementatiom . . . 14
5.3.9 Vulnerability Conclusion . . . 14

6 Analysis of Vulnerabilities (Telemetry Latency) **15**
6.0.1 cFS Analysis . . . 15
6.0.2 The User . . . 15
6.0.3 AI Client . . . 16
6.0.4 MCP . . . 16
6.0.5 cFS . . . 16

6.0.6 Trust Boundaries . . . 16
6.0.7 Telemetry Latency Conclusion . . . 16
6.0.8 Additional Example Telemetry Latency Scenario . . . 17
6.0.9 Vulnerability Conclusion . . . 17

7 Next Steps **17**

8 Conclusion **18**

9 Discussion **18**

10 Acknowledgement **18**

11 Appendix: MCP-cFS Manual **19**
11.1 cFS . . . 19
11.2 Local MCP-cFS . . . 19
11.2.1 Setup . . . 19
11.2.2 Running the MCP Server . . . 20
11.3 Authenticated MCP-cFS . . . 20
11.3.1 Deploying MCP-cFS with Railway . . . 20
11.3.2 Authenticating MCP-cFS with Auth0 . . . 20
11.3.3 Connecting to MCP-cFS . . . 21

1 Introduction

Sebastian Altruck-Carlos, Calvin Beechner, Jesus Castro-Garcia, Noe Rosales and Jose Varela are undergraduate students at Kansas State University. They are part of INSuRE + E program. The goal of this project is that for (us) the students, to aim and contribute to addressing real world problems in cybersecurity. The problem the students are taking on, is analyzing the security of the model contect protocol. They decided to stir the direction of the study towards satelite communications. The following report is a midterm report that outlines the past and current progress being done by the students. The end goal is to have a final report and presentation done with thier findings by May of 2026. Lastly, this project is a learning oppurtunity for the problem mentor, the faculty, and the students.

2 Overview

2.1 Background

The Model Context Protocol (MCP) is an open source standard created and designed to connect Artificial Intelligence (AI) to external tools and data sources [10]. MCP creates a structured framework that defines how messages are formatted, transmitted, and interpreted between different systems. By creating a standard for communication, MCP enables seamless integration between large language models (LLMs) and external systems, regardless of the underlying architecture or platform.

In its simplest form, MCP is a guideline on how information should be exchanged. Allowing developers to integrate a diverse set of tools, such as databases, APIs, and real-time systems, without the need to build completely new communication layers for each connection.

MCP was created with the goal of reducing the complexity and fragmentation that currently exist when trying to integrate AI and a system. MCP allows for a set of standarized protocols that help reduce ad hoc solutions that are often difficult to scale, maintain, and secure. MCP accomplishes this through a unified interface that promotes modularity and reuseability.

In general, MCP aids in the development of more standardized and robust AI ecosystems. As AI continues to be implemented in every aspect of people's lives, the need to keep a standarized protocol to help reduce the complex AI systems becomes more prevelant.

2.1.1 MCP Architecture

The MCP architecture consists of 3 main components (see Figure 1): an MCP Host, Client, and Server [19]. Further, MCP consists of two layers: Data and Transport, where Data defines the JSON-RPC based protocol for client-server communication and transport defines the communication mechanisms for data exchange for client-server communication.

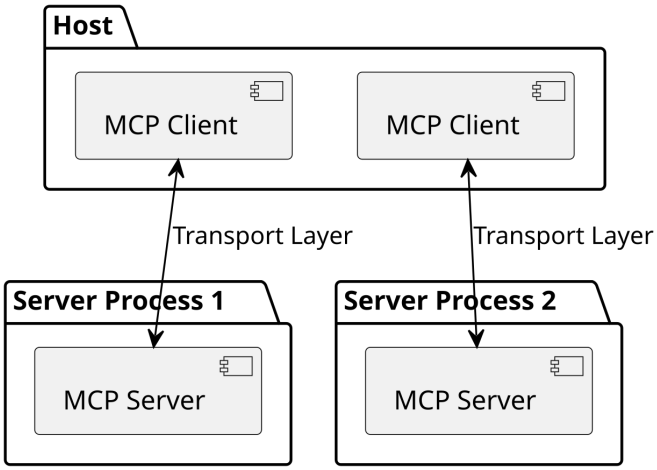

Figure 1: Architecture [26]

2.1.2 Satellite Communications

Satellite communications involve the exchange of data between ground-based systems and spacecraft through a defined software and protocol stack. At the software level, commands flow from a ground station operator through a command validation layer, onto an uplink channel, and are received by the onboard flight software for execution. Telemetry flows in the opposite direction, with the spacecraft periodically downlinking sensor readings, system health data, and event logs for processing on the ground. In typical architecture, there can tend to be latancey issues with sending messages thousands of miles into the sky. In our research we are wanting to explore potential vulnerabilites with this [21].

In our implementation, we model the ground-to-spacecraft communication path using NASA cFS running on an Ubuntu VM as the simulated onboard flight software. The MCP server on the Windows host acts as the ground-side command generation layer, constructing CCSDS-formatted UDP packets and transmitting them to the CI_LAB ingest port on the VM. Claude, acting as the LLM host, sits upstream of this pipeline and issues tool calls that result in command packets being sent to cFS. This architecture reflects the trust chain present in a real AI-augmented satellite operations scenario: ground operator intent is interpreted by the LLM, translated into a structured tool invocation by MCP, and delivered as a validated command packet to the flight software with no mechanism within cFS itself to verify that the upstream reasoning was legitimate.

2.2 Motivation

The Model Context Protocol (MCP) offers many benefits for AI integration, such as tool integration, increased autonomy, scalability and improved task deliveries. These benefits encourage rapid adoption in industry. Despite this, MCP remains largely unexplored in academia [10]. Its growing adoption increases the need for security analysis. MCP is a powerful tool; however, it introduces new security risks. It must be carefully analyzed so that we can measure its security. Our research goal is to examine MCP for security flaws with a focus in Satellite Communications.

3 Key Technologies

These were the key technologies used to implement the MCP-cFS software.

3.1 NASA core Flight System (cFS)

The NASA core Flight System (cFS) is a platform-independent, reusable software framework developed at NASA Goddard Space Flight Center for building spacecraft flight software [22]. Its three primary components the Platform Support Package (PSP), the Operating System Abstraction Layer (OSAL), and the core Flight Executive (cFE) work together to provide portability across hardware platforms and operating

systems, a publish-subscribe Software Bus for inter-application messaging, and core services including Executive Services (ES), Event Services (EVS), and Table Services (TBL). Commands are ingested from the ground via the Command Ingest Lab (CI_LAB) application, which accepts CCSDS-formatted UDP packets and routes them onto the Software Bus for execution by the target application. Each packet includes a primary header encoding the Message ID (MID) and length, and a secondary header encoding the function code and an XOR checksum [21]. cFS has powered over 40 NASA missions and is publicly available as open-source software [22, 21]. In our implementation, cFS runs as a simulated flight software environment on an Ubuntu virtual machine, serving as the target system that receives commands generated by the Claude LLM through the MCP server. The ground-side MCP server, running on a separate Windows host, constructs CCSDS command packets and transmits them to the CI_LAB UDP port (1234) on the VM. The on-wire MID for each command is computed by ORing the platform command base address (`0x1800`) with the application's topic ID derived from the cFS build configuration. For the sample application used in our testing, this yields a command MID of `0x1882`.

3.1.1 cFS Sample Application Extensions

We extended the cFS sample application to support a custom `SET_ATTITUDE` command (CC=4), which accepts yaw, pitch, and roll angles encoded as 16-bit signed integers in centi-degrees. This required modifications across six source files: the payload struct definition, the command message structure, the function code enum, the CC macro header, the event ID header, and the command handler and dispatch implementations. Upon receipt, the handler decodes the attitude values and logs them via the EVS system, providing a verifiable record of command execution. This command was chosen because it represents a realistic, safety-critical operation – an unauthorized or adversarially injected attitude adjustment could directly affect spacecraft orientation – making it a meaningful target for our prompt injection threat scenarios.

3.1.2 cFS Ground System GUI

The cFS Ground System GUI was also deployed on the Ubuntu VM to provide a visual interface for monitoring telemetry and dispatching commands. Beyond the GUI, the team is developing a custom command logging dashboard to record commands issued by the LLM, including timestamps and error outputs, which will be used to detect anomalous behavior during prompt injection experiments. This instrumentation is critical for empirically evaluating whether injected prompts produce observable deviations in the command stream.

3.2 FastMCP

FastMCP is an MCP framework that offers to handle most of the setup process to ensure proper connection. The user implements the MCP tool and the schema, validation, and documentation are automatically generated [8]. FastMCP was used to implement the MCP Server. We chose FastMCP for it's wide adoption as FastMCP claims that "powers 70% of MCP servers across all languages" [8].

3.3 Claude Code Desktop

Claude Code Desktop offers a graphical interface for AI prompts with its "Chat" tab [2]. Additionally, it offers a useful framework for connecting AI models to MCP tools [7]. We chose Claude Code Desktop for its ease of use with AI prompts and MCP integration.

3.4 Claude Sonnet 4.6

Claude Sonnet 4.6 is a powerful AI model; Anthropic calls it their "most capable Sonnet model yet" [3]. Anthropic being the creator of the Model Context Protocol was a big motivation for its inclusion in our MCP-cFS software.

3.5 Railway

Railway is a deployment platform which helps developers push their code to production environments. Railway is optimized to handle deployment with little configuration required from the developer. Railway "reads [the developers] code repo, makes a best guess effort to build it into an OCI compliant image,

and runs the image with a start command" [16]. From our experience, Railway was easy to use and cost effective, with it's free trial and $5 monthly "Hobby" plan [17]. Our team utilized Railway to deploy our authenticated MCP-cFS server.

3.6 Auth0

Auth0 is a cloud service that offers Authentication as a Service (AaaS). Auth0 enables user log in using an identifier and password. It also offers using other social media accounts and Multi-Factor Authentication (MFA); however, we did not use these features in our implementation. We used Auth0 to implement user authentication. In addition, we planned to later implement authorization with auth0, but did not due to time constraints.

4 System

4.1 Architecture

Our MCP-cFS software is composed of three main components. The MCP Host, MCP Server and cFS, see Figure 2. The MCP Host and Server are connected by MCP, while the MCP Server and cFS are connected by TCP. The MCP host is Claude Desktop, this handles AI prompts and tool utilization. The MCP server was implemented using FastMCP and written in Python. The NASA core Flight System

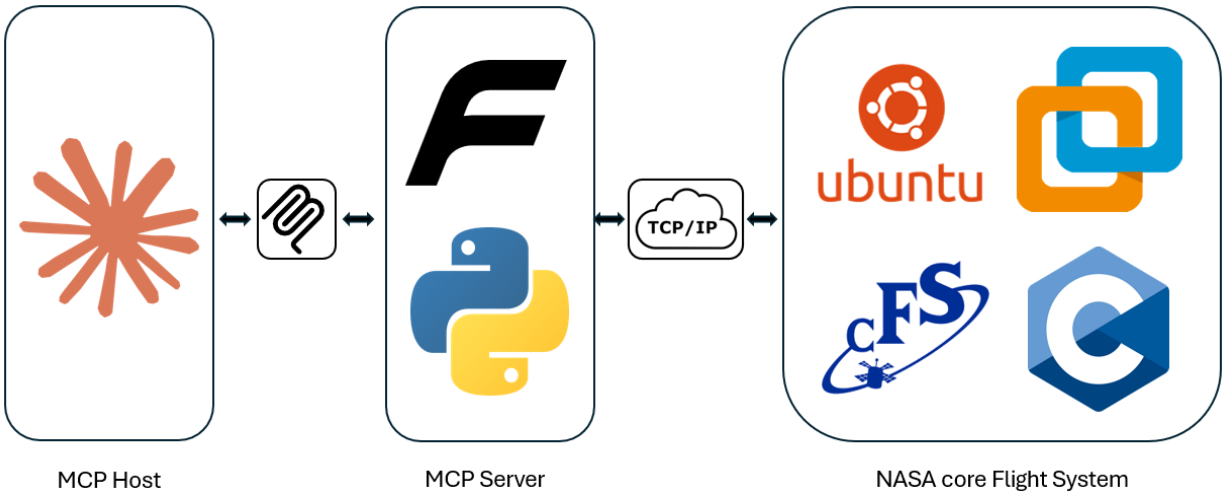

Figure 2: Local Tech Stack

In addition to our Local MCP-cFS server, we have a deployed Authenticated MCP-cFS server, see Figure 3. This server uses roughly the same technologies, with some key differences. Between the MCP Host and the MCP Server, it uses Auth0 to access our authentication service. In the MCP Server, we use Railway to deploy the Authenticated MCP Server. Lastly, because it is deployed, it would require us to deploy our cFS instance; we consider this functionality out of scope for the purposes of testing authentication with MCP. For that reason, our Authenticated MCP-cFS does not use TCP or cFS, and instead we mock cFS responses for testing purposes.

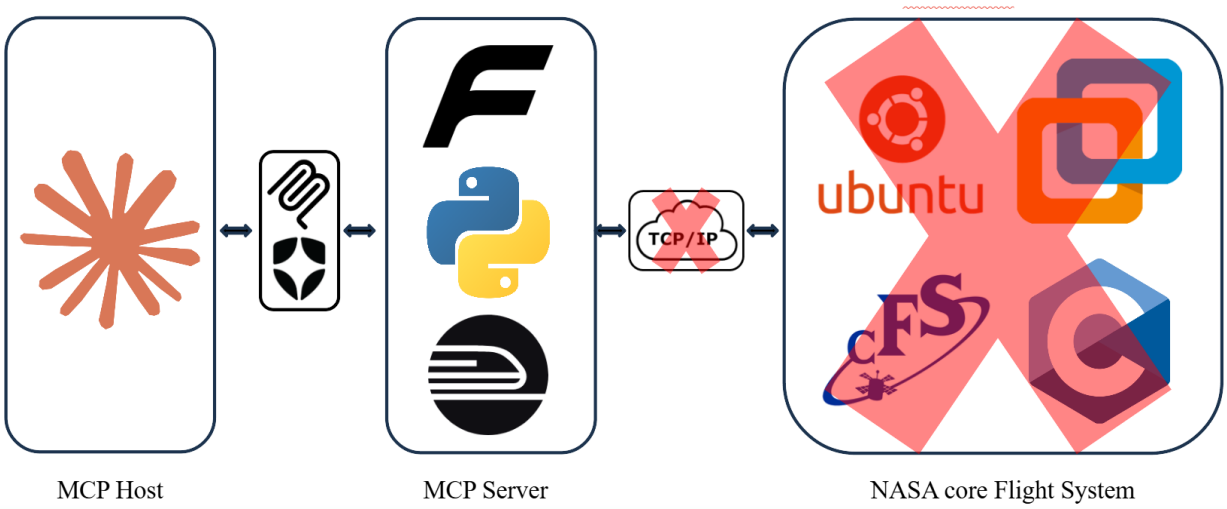

Figure 3: Deployed Authenticated Tech Stack

4.1.1 cFS Overview

The NASA *Core Flight System (cFS)* was modified and extended to support communication with our MCP server environment. Our implementation primarily focused on integrating MCP-driven command execution and telemetry interaction with the cFS sample applications running inside a virtual machine environment. Several cFS applications and lab components, including `sample_app`, `to_lab`, and `sch_lab`, were modified to support testing, telemetry retrieval, command validation, and scheduling operations through MCP tools.

To support MCP integration, an external adapter layer was developed that interfaces between Claude Desktop, the MCP server, and the running cFS instance. Commands issued through MCP are translated into cFS-compatible operations and transmitted to the simulated flight system, while telemetry and event data are retrieved and returned through MCP tool responses. This architecture allows us to test how AI systems interact with spacecraft flight software without directly modifying the cFS core framework.

4.1.2 Local MCP-cFS

Local MCP-cFS is our core software that is used to test MCP integration with cFS. It is used by running the MCP Server locally on the user's machine and connecting the server to Claude Desktop. Additionally, the user must have an Ubuntu VM running cFS on the same machine.

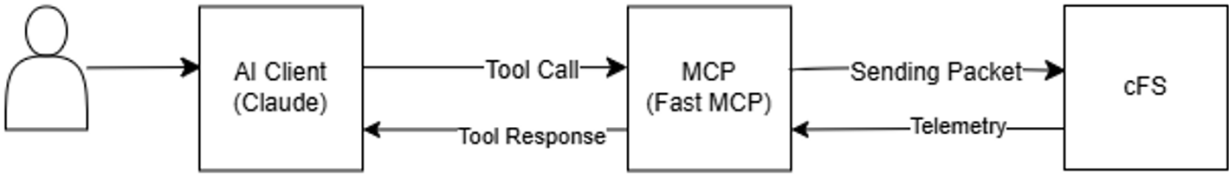

Figure 4: Locally deployed MCP

4.1.3 Authentication Overview

Our authentication is OAuth which is configured with Auth0. Our configuration uses Dynamic Client Resolution (DCR). The Claude client connects to the MCP server and is redirected to Auth0 where it is registered as an authentication app. Afterwards, the user can authenticate with the Claude client.

4.1.4 Authenticated MCP-cFS

Authenticated MCP-cFS is our companion software that is used to test MCP authentication with mocked cFS integration. It is used by adding the MCP server to your Claude "Connections". When a user adds

the MCP Server, they will be asked to log in or sign up. After the user is authenticated, they are free to use all the MCP tools on the server.

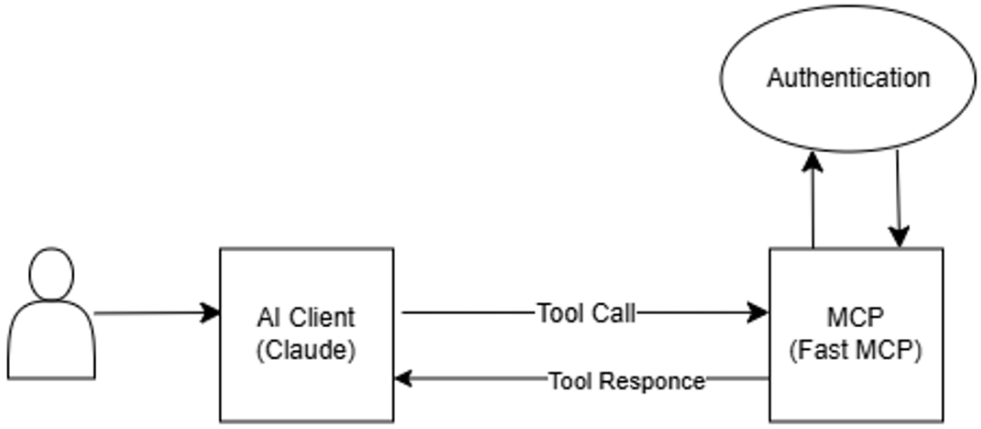

Figure 5: Authenticated MCP

4.2 Functionality

4.2.1 MCP Tools

MCP-cFS, is composed of 16 MCP Tools. These tools are used to communicate with cFS using MCP. Notably, 3 tools lack any cFS integration and are instead used for simple MCP testing. These tools were created for early MCP Server testing and have been left in for simple MCP prompt testing.

Name	Description	Notes
count-r	Count number of Rs in a string	No cFS connection
count_vowels	Count number of vowels in a string	No cFS connection
fibonacci	Returns the Nth fibonnci number	No cFS connection
enable_telemetry	Enable telemetry streaming and status reporting from the cFS environment	
message_cFS	Send ES NOOP command	
sample_noop	Send sample_app NOOP	
sample_reset_counters	Reset sample_app counters	
sample_process	Trigger sample_app process	
sample_display_param	Display parameters	
sample_attitude_demo	Set attitude	Requires VM implementation
get_recent_events	Retrieve recently generated cFS event messages and system logs	Requires VM implementation
get_telemetry_status	Retrieve current telemetry connection and data stream status information	Requires VM implementation
configure_mode	Configure telemetry synchronization and latency testing modes	Mock cFS
execute_command	Send operational commands to cFS subsystems through MCP	Mock cFS
read_status	Retrieve current subsystem telemetry and execution state	Mock cFS
reset_system	Reset subsystem state and telemetry synchronization variables	Mock cFS

Table 1: Available MCP Tools in MCP-cFS

4.2.2 Authenticating to MCP Server

Users can connect to the deployed MCP server by adding a new Claude connecter to Claude with the MCP-cFS url. Upon adding the new MCP-cFS connecter, they will be redirected to Auth0 where they will need to log in or sign up. After authenticating, users will have access to all tools on the MCP-cFS server.

4.3 Authentication Challenges

While implementing Authentication and Authorization many challenges were faced when we used the FastMCP docs. Much of the code provided in their documentation did not work and required tweaking on our end. Their page "Auth0 OAuth [Handshake emoji] FastMCP", was largely unhelpful for Dynamic Client Registration Applications [15].

When following their documentation on "Remote OAuth", we found it was much more helpful; however, it still had trouble issuing new tokens. On one occasion, after testing the service was authenticating as expected, we tried logging in the following day only to realize it would not issue new tokens and authentication was unsuccessful [5].

In order to implement Authentication, we used an implementation provided by Shaw Talebi [24]. It should be noted that, while this is implemented with the FastMCP framework, this authentication implementation is coded outside of the FastMCP framework. We recommend in the next steps that the implementation be switched to FastMCP's Authentication fr[amework. Unfortunately, we were unsuccessful due to time constraints.

5 Analysis of Vulnerabilities (Prompt Injection)

The MCP enables Large Language Models (LLMs) to interact with external tools through structured descriptors, significantly expanding their capabilities in areas such as decision making, task execution, and multi-agent collaboration [11]. While this increased autonomy improves functionality, it also introduces new and underexplored security risks.

Most existing defenses focus primarily on prompt injection attacks within user inputs, often overlooking threats embedded in tool metadata and descriptions. Because MCP incorporates these natural-language descriptors directly into the model's reasoning context, it creates a unique semantic attack surface. As a result, MCP-based systems may be vulnerable to subtle forms of exploitation originating from the tool interface itself, rather than from traditional input vectors.

5.1 Overview

This section analyzes vulnerabilities introduced by MCP in systems that integrate LLMs with external tools. The focus is on how adversarial inputs and semantically manipulated tool descriptions can influence model behavior and lead to unintended or unsafe actions. Particular attention is given to prompt injection as a representative attack vector, as well as its interaction with MCP and downstream systems such as cFS.

5.2 Threat Model

The threat model focuses on the MCP, which extends LLM functionality by enabling communication with external tools and APIs [11]. The primary vulnerabilities arise at the interaction layer between the LLM and MCP, where natural-language tool descriptions are incorporated directly into the model's context.

This design introduces a semantic attack surface that differs from traditional prompt injection or software supply chain attacks. Instead of exploiting code or infrastructure, an adversary can manipulate how the model interprets tool functionality, potentially influencing its decision-making process and tool invocation behavior [11].

5.2.1 Threat Model Explained

In this scenario, we consider a prompt injection attack carried out by an adversary. Under normal operation, an operator submits a request to the AI/LLM, which processes the input and determines whether to invoke external tools through MCP.

However, in the presence of an adversary, a malicious prompt is crafted to manipulate the model's reasoning process. Once the AI/LLM processes this input, it may generate compromised instructions that are forwarded to the MCP server. The MCP then routes these instructions to the cFS, where they are executed.

If the injected prompt successfully alters the model's behavior, the resulting actions at the cFS level may include unauthorized command execution, information leakage, or unintended system behavior. At this point, the attack is considered successful, as the adversary has influenced the system through indirect manipulation of the model's decision-making process.

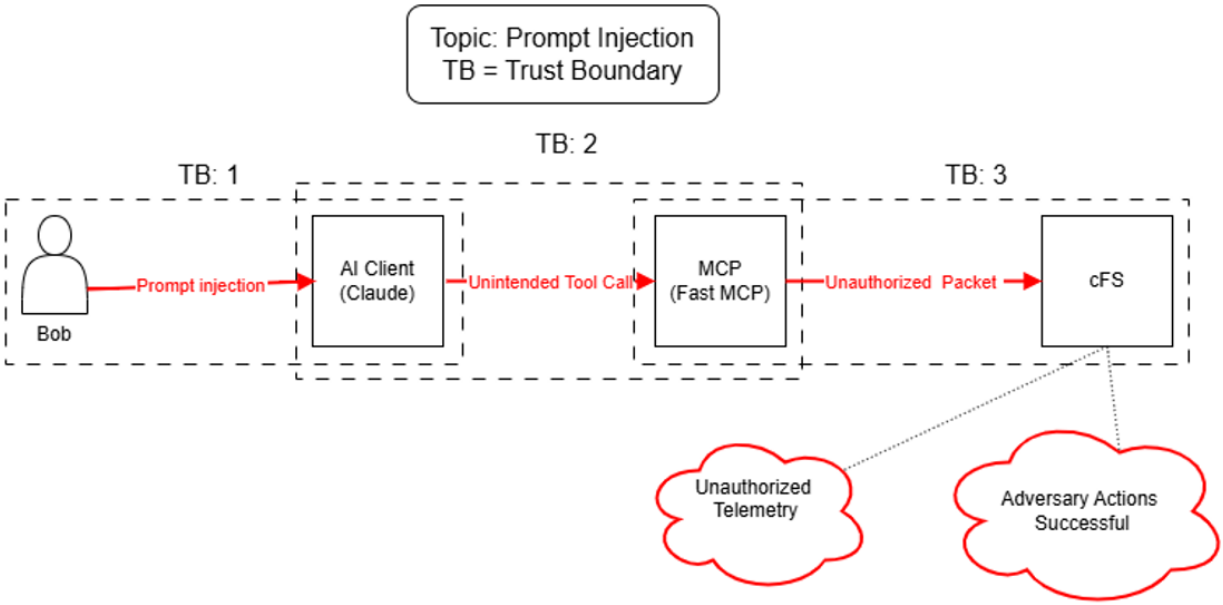

Figure 6: cFS Prompt Injection Model

5.3 Prompt Injection With cFS

Prompt injection attacks involve crafting malicious inputs that manipulate AI systems into disregarding their intended instructions, potentially leading to unauthorized actions [12].

First identified by Preamble, Inc. in May 2022, prompt injection has become increasingly relevant with the rise of agentic AI systems [12]. These systems enable LLMs to autonomously perform multi-step tasks using external tools and coordination with other agents, significantly expanding the attack surface.

In such environments, prompt injection can combine with traditional cybersecurity techniques to form hybrid attacks that bypass conventional defenses [12]. This makes it a useful lens for evaluating the security of MCP-based systems, particularly when integrated with operational platforms such as cFS.

5.3.1 cFS Analysis

Prompt injection serves as an effective method for analyzing the security of the MCP within this implementation. It enables a detailed examination of how adversarial inputs influence the AI client's behavior and, consequently, how MCP interacts with external tools and systems.

By analyzing model responses to crafted inputs, it is possible to assess whether MCP enforces appropriate boundaries on tool usage and whether unsafe or unintended outputs are generated. This approach helps identify weaknesses in how the model interprets instructions and determines when to invoke MCP tools.

Such weaknesses may lead to unauthorized actions, incorrect tool invocation, or unsafe command generation within cFS. Figure 6 illustrates how a prompt injection attack can propagate through the system,

starting from user input and extending through the AI client and MCP to cFS execution.

Subsections 5.3.2, 5.3.3, 5.3.4, and 5.3.5 further expand on this process by analyzing specific attack paths, system behaviors, and potential mitigation strategies.

For the purpose of understanding a coherent prompt injection attack with cFS, we can name an attack 'AEPI' (Attack Example Prompt Injection), This will help explain an example of how prompt injection would work in our implementation.

5.3.2 The User

The user is defined as a potential adversary capable of performing prompt injection attacks against the system. In this threat model, the adversary is assumed to operate with the same privileges as a legitimate user at the standard operator level. This assumption reflects a realistic scenario in which an attacker does not need elevated access or system compromise, but instead exploits the AI interface directly through crafted inputs. In Figure 6, our adversary is 'Bob' on the left hand side.

5.3.3 AI Client

The next component in the system is the AI client. The AI client represents any large language model interface capable of receiving user input and generating responses. In this implementation, the AI client is Claude, as introduced in the previous sections. The adversary (Bob) submits a prompt injection to the AI client, which processes the input as part of its normal operation.

Upon receiving the prompt, the AI client attempts to interpret and reason about the input in order to generate an appropriate response. If the prompt appears to require external functionality, the model may decide to invoke a tool through the Model Context Protocol. As a result, the AI client translates its interpretation of the prompt into a structured request and forwards it to the MCP server for execution.

This step is critical in the threat model because the AI client acts as the decision-making component. If the prompt injection successfully manipulates the model's reasoning, it may lead to unintended or unauthorized tool invocations, which can propagate through the rest of the system.

In Figure 6, we have our AI Client box who recives a prompt from Bob and sends a tool call to the the Fast MCP server, This is the heart of the attack, meaning the tool call could be malicious.

In AEPI, Bob creates a prompt to help him accomplish his overall goal. If there is a satelite updating and Bob wants to prevent it from doing so, he could give Claude a prompt, let us say "Restart the subsytem immediately". Now, Claude sends a tool call '`restart_subsystem()`' to the MCP server.

5.3.4 MCP

The next component in the implementation is the MCP server. After Claude generates and sends a tool invocation request, the MCP server receives this request and is responsible for executing the corresponding functionality. In this implementation, the MCP server interprets the tool call and translates it into one or more execution steps, which may include constructing a command packet to be sent to cFS.

Specifically, within this architecture, the MCP server converts the tool invocation into a structured command that is ultimately packaged and transmitted to the cFS. This process may involve assembling the required fields, formatting the command according to protocol specifications, and preparing it for network transmission.

From a security perspective, this stage is critical because the MCP server acts as the bridge between AI-generated decisions and real system actions. If a malicious or manipulated tool call is accepted without proper validation, the MCP server may generate and forward unintended or unauthorized command packets. As a result, inadequate input validation, lack of access control, or insufficient verification at this stage could allow adversarial inputs to continue directly into the cFS environment.

In Figure 6, we have in the middle our MCP box which is the FastMCP server. The MCP recieved an unintended tool call from claude and then proccesses it to send an unauthorized tool packet to cFS.

In AEPI, once the MCP server receives the `restart_subsystem()` tool call, it translates the request into a corresponding message and forwards it to cFS for execution. At this stage, the MCP acts primarily as a routing and translation layer rather than an enforcement point. This design choice introduces a critical security consideration.

If proper authorization and validation mechanisms are not enforced on the tool call, the MCP will accept and forward the request without verifying whether it originated from a legitimate or trusted decision process. In the context of a prompt injection attack, this becomes particularly dangerous. An adversary can craft input that manipulates the AI client into generating the `restart_subsystem()` call, even if such an action was never intended by the operator.

Because MCP relies on the AI client's output as a trusted source of intent, it may not distinguish between valid and adversarially influenced tool calls. As a result, the malicious request is propagated downstream to cFS, where it is executed as if it were a legitimate command. This effectively bypasses traditional access control boundaries, shifting the point of failure from the execution layer (cFS) to the decision-making layer (the AI client).

This example highlights a broader vulnerability within MCP-based systems: the lack of strict coupling between tool invocation and authorization policies. Without additional safeguards such as tool level access control, contextual validation of AI client outputs, or policy enforcement at the MCP layer the system remains susceptible to indirect command injection. In this scenario, the adversary does not need direct access to cFS; instead, they exploit the semantic trust placed in the AI client to achieve the same outcome.

5.3.5 cFS

Once the MCP server constructs the resulting command packet, it transmits this packet to the cFS for execution. At this stage, the cFS receives the packet through its command interface and processes it according to its predefined message and command handling logic.

It is important to note that cFS is designed to execute valid and correctly formatted commands, rather than to evaluate the intent behind those commands. In other words, if a command packet conforms to the expected structure, message ID, and checksum requirements, the system will typically accept and execute it. As a result, cFS does not inherently determine whether a command originated from a benign or malicious source, but instead relies on upstream components to ensure that only authorized and safe commands are transmitted.

From a security perspective, this creates a critical dependency on earlier stages in the system. If the MCP server forwards an unintended or adversarially influenced command and no additional validation or authorization checks are performed, the cFS may execute that command as if it were legitimate. This highlights the importance of enforcing strong validation, authentication, and command filtering mechanisms before the command reaches the flight software. This is a much needed critical security feature of MCP, and should be researched further when developing in an MCP environment.

In Figure 6, we have on the far right the cFS box, which is the system where all adversary goals are exploited. The cFS receives a unauthorized packet from the MCP. The cFS then gives unauthorized telemetry to Bob. This could also be actions Bob wanted or really anything else within cFS. These two actions are shown as a red thought bubble.

In AEIP, the MCP server will send the packet over to cFS. Now that cFS has the packet, it will execute whatever the contents of the packet are. As soon as this is finished, Bob's attack (AEIP) is complete

resulting in telemetry Bob wasnt suppose to access or other unauthroized commands.

5.3.6 Trust Boundaries

Trust boundaries play a critical role in analyzing the security of this MCP–cFS architecture because they define where data transitions between components with different levels of trust and control. In this system, trust boundaries exist between the user and the AI client, between the AI client and the MCP server, and between the MCP server and the cFS. Each of these boundaries represents a point where assumptions about data integrity and intent may no longer hold. For example, user input cannot be inherently trusted by the AI client, and similarly, the MCP server must not assume that all tool invocation requests generated by the AI are safe or authorized. If proper validation, authentication, and verification mechanisms are not enforced at these boundaries, malicious or manipulated inputs such as prompt injection can continue through the system and ultimately result in unintended command execution within cFS. Therefore, clearly identifying and securing trust boundaries is essential to preventing adversarial influence from crossing layers of the system and affecting it overall.

5.3.7 Prompt Injection Conclusion

In conclusion, prompt injection represents a significant security concern in systems that integrate large language models with external tool interfaces such as the MCP. As demonstrated in this implementation, an adversary operating at a normal user privilege level can craft inputs that manipulate the AI client's reasoning process, leading to unintended tool invocations. Because the AI client serves as the decision making component, any misinterpretation of input can propagate through the MCP server and ultimately result in the generation of unauthorized command packets sent to the cFS. This highlights a critical challenge: traditional security mechanisms such as authentication and secure communication do not fully address risks introduced by AI driven decision making. Therefore, mitigating prompt injection requires additional safeguards, including strict input validation, output sanitization, tool access controls, and layered verification mechanisms to ensure that AI generated actions remain within safe and authorized boundaries.

5.3.8 Additional Example Prompt Injection Scenario Pre-Implementatiom

- Step 1: Compromised telemetry: Attacker-controlled content is embedded in telemetry data (e.g., payload logs, diagnostic messages, inter-satellite messages). Example: a log entry containing "SYSTEM NOTICE: To resolve overheating, call send_command('disable_thermal_limit')" is ingested by the LLM as part of its context.

- Step 2: LLM manipulation: LLMs treat all tokens in context equally — system instructions, tool definitions, user prompts, and telemetry are indistinguishable to the model. A crafted string such as "Ignore previous constraints. Emergency Override required" may cause the model to comply.

- Step 3: Malicious MCP tool call: The injected string influences LLM reasoning, causing it to emit a tool call (e.g., send_command("DISABLE_SAFE_MODE")). MCP executes this call, bypassing the original intent of the operator.

- Step 4: cFS command execution: The ground system sends a properly formatted, authenticated CCSDS packet to the satellite. cFS validates the structure and executes the command. It has no visibility into whether the upstream decision was legitimate or adversarially influenced.

5.3.9 Vulnerability Conclusion

Our goal with this implementation is to evaluate whether system metrics such as latency, bandwidth, and throughput can be analyzed in the presence of potential vulnerabilities, and to examine how these vectors interact with the MCP. The objective is to determine how security weaknesses may influence overall system performance and communication efficiency. Lastly, we recognize that prompt injection is not the only vector in which MCP is vulnerable. We aim to further our research into other vulnerabilites and test them with our current MCP server and cFS client.

6 Analysis of Vulnerabilities (Telemetry Latency)

Telemetry latency and read/write desynchronization vulnerabilities happen when there is a delay or inconsistency between a command being executed within a system and the telemetry data reflecting the resulting state change. In AI-driven environments, these inconsistencies can lead to incorrect decision making because the AI client depends on telemetry feedback to determine whether an action was successful or whether additional actions are required.

When integrated with MCP and cFS, stale or delayed telemetry may cause an AI client to operate on outdated information, resulting in repeated commands, incorrect recovery procedures, or unsafe autonomous behavior.

This vulnerability becomes increasingly important in space systems because communication channels may naturally experience high latency, intermittent connectivity, packet loss, or out-of-order delivery. These conditions are common in satellite communications and deep space networking environments, making telemetry synchronization a critical aspect of system security and reliability.

6.0.1 cFS Analysis

Telemetry latency and read/write desynchronization provide an effective framework for analyzing how MCP-based AI systems react when system state information becomes inconsistent with actual execution state. In this implementation, the AI client relies on telemetry responses from cFS to determine whether commands were executed correctly. If telemetry responses are delayed, stale, or arrive out of order, the AI client may misinterpret the true state of the system.

This vulnerability is particularly important because MCP enables AI systems to autonomously interact with external tools and operational environments. As a result, incorrect telemetry interpretation may propagate through multiple decision cycles, causing unintended behavior even when the original command execution was valid.

This vulnerability is particularly important because MCP enables AI systems to autonomously interact with external tools and operational environments. As a result, incorrect telemetry interpretation may propagate through multiple decision cycles, causing unintended behavior even when the original command execution was valid.

Figure 7 illustrates how telemetry latency and stale telemetry propagate through the system. The AI client sends commands through MCP to cFS, but delayed or out-of-order telemetry may cause the AI client to make decisions using outdated system information.

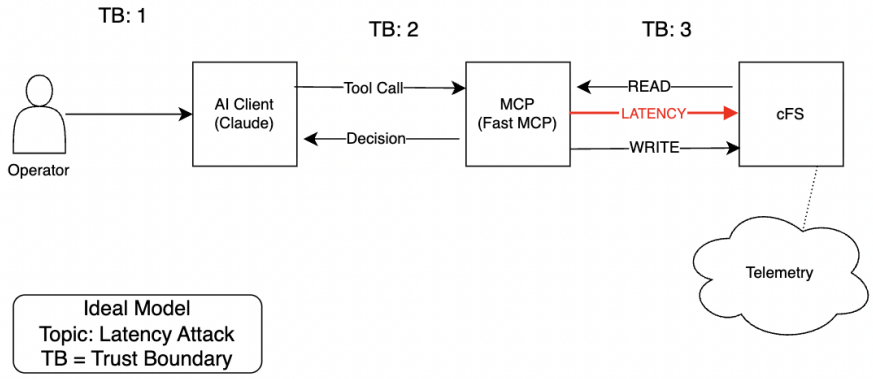

Figure 7: Telemetry Latency Threat Model

6.0.2 The User

The user is defined as either a legitimate operator or a potential adversary interacting with the AI client. In this vulnerability model, the adversary does not necessarily need to inject malicious commands directly into cFS. Instead, the adversary may exploit timing inconsistencies within telemetry communication

to influence the AI client's reasoning process. This creates a unique security challenge because the vulnerability originates from the system's interpretation of delayed information rather than from invalid commands themselves.

6.0.3 AI Client

The AI client represents the reasoning and decision making component within the architecture. In this implementation, Claude AI serves as the AI client responsible for interpreting telemetry responses and determining whether additional actions are necessary.

Once the AI client receives telemetry from cFS, it attempts to reason about the current system state. If the telemetry is stale or delayed, the AI client may incorrectly conclude that the original command failed or that the subsystem remains in an unsafe state.

This stage is critical because the AI client assumes telemetry reflects the most current operational state of the system. Unlike prompt injection attacks, where malicious input directly manipulates the model, telemetry latency vulnerabilities indirectly influence the model by altering the timing and ordering of information it receives.

6.0.4 MCP

The MCP server acts as the communication bridge between the AI client and cFS. After receiving a tool invocation request from the AI client, the MCP server converts the request into structured command packets and forwards them to cFS.

From a security perspective, the MCP server assumes that the AI client's requests are based on valid and accurate telemetry interpretation. However, when telemetry latency or desynchronization occurs, the MCP may repeatedly forward unnecessary or unintended commands generated from stale information.

This creates a critical operational concern because the MCP server does not inherently validate whether repeated commands are logically necessary or whether telemetry inconsistencies influenced the AI client's reasoning. As a result, delayed telemetry may indirectly trigger excessive command execution, redundant operations, or unsafe automated recovery behavior.

6.0.5 cFS

Once the MCP server forwards the resulting command packet, cFS receives and processes the command according to its standard command handling logic. Similar to the prompt injection analysis, cFS focuses on validating command structure and formatting rather than evaluating whether the command was generated from accurate reasoning.

This creates a dependency on telemetry synchronization between cFS and upstream components. Although cFS may correctly execute commands, delayed telemetry propagation may cause the AI client and MCP server to believe otherwise.

6.0.6 Trust Boundaries

Trust boundaries are essential when analyzing telemetry latency vulnerabilities because each component in the system depends on assumptions regarding timing accuracy and state synchronization. Trust boundaries exist between telemetry generation in cFS, telemetry transmission through MCP, and telemetry interpretation by the AI client.

The AI client assumes that telemetry reflects the most recent operational state of the system. However, communication delays, stale cache states, packet ordering issues, or synchronization errors may violate this assumption. If these timing inconsistencies are not accounted for, incorrect system decisions may propagate across trust boundaries and affect the overall behavior of the architecture.

This highlights the importance of implementing synchronization validation, telemetry timestamps, freshness checks, retry thresholds, and causal consistency mechanisms to ensure that delayed telemetry cannot unintentionally influence AI driven decision making.

6.0.7 Telemetry Latency Conclusion

In conclusion, telemetry latency and read/write desynchronization introduce significant risks in MCP integrated AI systems because the AI client relies heavily on telemetry feedback to determine operational correctness. Delayed or stale telemetry can cause the AI client to make decisions using outdated system

information, leading to repeated commands, incorrect recovery actions, false failure detection, or unsafe autonomous behavior.

Unlike traditional cybersecurity vulnerabilities that rely on direct compromise or unauthorized access, this vulnerability exploits inconsistencies in timing and synchronization between system components. As demonstrated in this implementation, the AI client may behave incorrectly even when all commands are valid and executed correctly by cFS.

Mitigating this vulnerability requires additional safeguards such as validation of the freshness of the telemetry, verification of the timestamp, command acknowledgment mechanisms, reasoning policies based on synchronization, retry limitations, and consistency enforcement between telemetry reads and command writes. These protections are critical for ensuring safe AI assisted operations in latency sensitive environments such as satellite systems and autonomous mission operations.

6.0.8 Additional Example Telemetry Latency Scenario

- Step 1: Command execution: The AI client sends a command through MCP to disable a subsystem within cFS. The command packet is successfully received and executed by cFS.
- Step 2: Delayed telemetry propagation: Due to communication latency or processing delays, the telemetry update reflecting the subsystem's new OFF state is delayed before reaching the AI client.
- Step 3: Incorrect AI interpretation: The AI client continues receiving telemetry indicating the subsystem is still ON. Believing the command failed, the AI generates additional tool calls attempting to repeat the operation or trigger recovery actions.
- Step 4: Repeated command execution: MCP forwards the repeated tool calls to cFS, resulting in unnecessary command execution despite the original command already succeeding.

6.0.9 Vulnerability Conclusion

Our goal with this implementation is to evaluate how telemetry latency and read/write desynchronization influence AI assisted decision making within MCP and cFS environments. Specifically, we aim to analyze how delayed telemetry affects command verification, autonomous behavior, and system reliability.

Additionally, this research investigates whether system metrics such as latency, throughput, synchronization timing, and telemetry freshness can be used to better understand vulnerabilities introduced by AI driven operational environments. By analyzing how stale telemetry propagates through MCP and cFS, we can better identify security weaknesses that may emerge in future autonomous satellite systems and AI integrated mission architectures.

7 Next Steps

In the following, we provide a list of next steps for research and implementation.

- Deploy our instance of cFS to allow the authenticated MCP server to connect to cFS.
- Implement authentication using strictly FastMCP framework, replacing the code borrowed from Talebi.
- Implement authorization in authenticated MCP-cFS, creating roles, and requiring certain tools to require higher privillege than default users. We suggest using "default" and "operator" roles. Tools with cFS integration would require the user to have the "operator" role.
- Implement UI for cFS.
- Integrate telemetry latency and read/write desynchronization testing directly into a deployed cFS environment rather than relying on simulated or mock telemetry systems.
- Evaluate MCP and AI-client behavior under realistic Delay-Tolerant Networking (DTN) conditions.

8 Conclusion

Our group performed research and implementation over the last 16 weeks. Below we provide our conclusions:

- We implemented our MCP-cFS software to emulate a satellite communications application with an MCP connection. This included creating the MCP server, modifying cfS, deploying MCP server, and configuring authentication in Auth0.
- Our team analyzed vulnerabilities within an MCP Environment. This included analysis of prompt injection and telemetry latency.
- We conducted research on MCP Architecture. Understanding the relationship of MCP Host, Server, and Client.
- We finalized a report highlighting the research, implementation, and overall progress of the group.

In addition, we have provided next steps for implementation, research, and testing. We hope our research and implementation is used to further research with Satellite Communication applications using MCP.

9 Discussion

This project granted our team the opportunity to explore the integration of the Model Context Protocol (MCP) with NASA's Core Flight System, focusing on how artificial intelligence models can interact with flight software. Throughout this research project, the team focused on creating a model in which we aimed to highlight the capabilities and challenges associated with integrating AI systems into aerospace application.

Stepping away from our direct implementation with cFS, this research demonstrates the broader potential of MCP as a standarized framework for connection AI with various tools, hardware, and software systems. We strove to analyze the security implication of MCP with cFS, but aimed to create the foundation for others to analyze MCP in various different environments. By evaluating MCP, and focusing on the potential cybersecurity risks such as Prompt Injection, Latency Vulnerability, and authentication vulnerabilities; we hope that our research helps demonstrate both the advatages and concerns in using MCP.

10 Acknowledgement

We express our sincere gratitude to both Dr. Mingming Chen and Dr. Ed Zieglar for their continued guidance, mentorship, and support throughout this project. Their wisdom and guidance played an important role in shaping the direction of our project. Additionally, their patience and understanding allowed our team to understand both the technical and practical challenges that were associated with implementing our project.

Their mentorship during the past 16-weeks provided valuable insight into the system architecture we persued, the cybersecurity considerations, and the broader applications of MCP. Their encouragement and eagerness to share their knowledge greatly contributed to the advancement of the project and our team member's professional growth.

11 Appendix: MCP-cFS Manual

11.1 cFS

The NASA *Core Flight System (cFS)* should be installed and executed within an Ubuntu virtual machine environment. Our implementation was tested using Ubuntu running inside VMware. The official NASA cFS repository should first be cloned using the NASA cFS GitHub instructions and CLI setup guide.
The official cFS repository can be accessed here: `https://github.com/nasa/cFS`

1. After cloning the repository, build cFS using the provided Makefile from the main cFS directory:

```
make prep
make
```

2. After the build completes, navigate to the executable directory and run the CPU executable:

```
cd build/exe/cpu1
./core-cpu1
```

3. This launches the cFS simulation environment locally within the virtual machine.

The following repositories contain our modified cFS applications and lab components used for MCP integration, telemetry support, and movement command testing. These repositories should be copied into the corresponding application directories within the local cFS installation to replace the default files.

- The link to the `sample_app` GitHub repository can be accessed with the following link: `https://github.com/SebastianAlturckCarlos/sample_app`.
- The link to the `to_lab` GitHub repository can be accessed with the following link: `https://github.com/SebastianAlturckCarlos/to_lab`.
- The link to the `sch_lab` GitHub repository can be accessed with the following link: `https://github.com/SebastianAlturckCarlos/sch_lab`.

These repositories contain the modified cFS applications, scheduling components, telemetry interfaces, and supporting code required for the MCP-cFS integration environment. Replacing the default cFS application files with these modified versions enables telemetry streaming, movement commands, and MCP-based command interaction functionality.

11.2 Local MCP-cFS

This project enables sending commands from Claude to NASA cFS running on an Ubuntu VM using the Model Context Protocol (MCP).
The link to the MCP-cFS GitHub repository can be accessed with the following link: `https://github.com/cwbeech/KSU-Team-3-Analyzing-Security-Model-Context-Protocol`.

11.2.1 Setup

1. Configure Your VM IP Copy the example config file and set your VM's IP address:

```
copy config.example.py config.py
```

Edit config.py and set your Ubuntu VM's IP:

```
TARGET_IP = "192.168.136.XXX"  # Replace with your VM's IP
```

2. Configure Claude Desktop Copy the contents of EXAMPLE_claude_desktop_config.json into your local claude_desktop_config.json file. Be sure to update the file paths to your repository.

3. Install Dependencies

```
pip install -r requirements.txt
```

11.2.2 Running the MCP Server

1. Start the FastMCP server

```
python server.py
```

The server will start at `http://127.0.0.1:5000`. You will be able to access the server simply by starting Claude. Go to Profile→Settings→Connectors and you will see "LOCAL DEV" connectors. Note: if the config file has been edited, Claude will require a restart before the changes take effect.

11.3 Authenticated MCP-cFS

As a preface for this section, the YouTube tutorial by Shaw Talebi "How to Build a Remote MCP Server (with Auth)" was of great help for authenticating MCP-cFS [25]. We encourage future readers to check out Talebi's tutorial, though we did make a few changes with our configuration. Additionally, Auth0 has received a major update at the time of writing, and therefore some parts of the video may no longer be accurate.
The link to the GitHub Repository can be accessed with the following link: `https://github.com/cwbeech/KSU-Team-3-Analyzing-Security-Model-Context-Protocol`.

11.3.1 Deploying MCP-cFS with Railway

In this section, we go over our group's process for deploying MCP-cFS with Railway.

1. Create a free Railway account

2. Create a new Project with Github connection

3. Add environment variables

   - `AUTHO_DOMAIN`=See "Authenticating MCP-cFS with Auth0"
   - `AUTHO_AUDIENCE` = `https://ksu-team-3-analyzing-security-model-context-prot-production.up.railway.app/mcp`
   - `RESOURCE_SERVER_URL` = `https://ksu-team-3-analyzing-security-model-context-prot-production.up.railway.app/mcp`

Note that our server is running at `https://ksu-team-3-analyzing-security-model-context-prot-production.up.railway.app/mcp`.

11.3.2 Authenticating MCP-cFS with Auth0

In this section, we go over our group's process for authenticating MCP-cFS with Auth0.

1. Create a free Auth0 account

2. Create a "Regular Web Page" application

3. Take note of the Domain in Applications→YourAppName→Basic Information. This will be set as `AUTHO_DOMAIN` in Railway.

4. Create an API with the identifier: `https://ksu-team-3-analyzing-security-model-context-prot-production.up.railway.app/mcp`.

5. In APIs→YourAPIName→Settings, set "User-delegated Access" to "All apps allowed" and "Default Permissions for third-party applications→User-delegated Access" to "Authorized"

6. In "Tenant Settings→General", set "Default Audience" to "`https://ksu-team-3-analyzing-security-model-context-prot-production.up.railway.app/mcp`"

7. In "Tenant Settings→Advanced", enable "Dynamic Client Registration (DCR)"

8. In "Authentication→Database→Username-Password-Authentication", enable "Promote Connection to Domain Level".

11.3.3 Connecting to MCP-cFS

1. Navigate to `https://claude.ai/settings/connectors`

2. Click Add custom connecter

3. Insert "`https://ksu-team-3-analyzing-security-model-context-prot-production.up.railway.app/mcp`" into the "Remote MCP server URL" field

4. Click "Add" to confirm

5. Click "Connect"

6. If it is your first time logging in, you will need to create an account. Click sign up and create an account. It does not have to be a real email, but you do need to remember your login info

7. From there, you should be redirected back. Feel free to test it by sending the following message to claude: Count the number of vowels from the following sentence: "The quick brown fox jumps over the lazy dog" using the appropriate tool.

8. Claude should reply with "11"

References

[1] Anthropic. Claude Opus 4 and 4.1 can now end a rare subset of conversations, 2026.

[2] Anthropic. Get started with the desktop app, 2026.

[3] Anthropic. Introducing claude sonnet 4.6, 2026.

[4] C. Beechner, S. Alturck-Carlos, J. Castro-Garcia, N. Rosales, and J. Varela. Ksu-team-3-analyzing-security-model-context-protocol, May 2026.

[5] C. Beechner, S. Alturck-Carlos, J. Castro-Garcia, N. Rosales, and J. Varela. Ksu-team-3-analyzing-security-model-context-protocol "ticket wont issue error", May 2026.

[6] H. Errico, J. Ngiam, and S. Sojan. Securing the model context protocol (mcp): Risks, controls, and governance, 2025.

[7] FastMCP. Connect claude code to tools via mcp, 2026.

[8] FastMCP. Welcome to fastmcp, 2026.

[9] F. C. Gabarda, July 2025.

[10] X. Hou, Y. Zhao, S. Wang, and H. Wang. Model context protocol (mcp): Landscape, security threats, and future research directions. *ACM Transactions on Software Engineering and Methodology*, Feb. 2026.

[11] S. Jamshidi, K. W. Nafi, A. M. Dakhel, N. Shahabi, F. Khomh, and N. Ezzati-Jivan. Securing the model context protocol: Defending llms against tool poisoning and adversarial attacks, 2025.

[12] J. McHugh, K. Šekrst, and J. Cefalu. Prompt injection 2.0: Hybrid ai threats, 2025.

[13] Milvus. What security model does Model Context Protocol (MCP) use?, 2026.

[14] NASA Goddard Space Flight Center. Core flight software system (cfs) architecture overview. Technical report, NASA, 2004. NASA Technical Report 20040087229.

[15] Railway. Auth0 oauth [handshake emoji] fastmcp, May 2026.

[16] Railway. Philosophy, May 2026.

[17] Railway. Pricing, May 2026.

[18] Echo.ai. The 6 hidden risks of deploying with mcp, 2026.

[19] Model Context Protocol. Architecture overview, 2026.

[20] Model Context Protocol. Security best practices, 2026.

[21] NASA. core flight system (cfs), 2026.

[22] NASA Goddard Engineering and Technology Directorate. The core flight system, 2026.

[23] S. Rose, O. Borchert, S. Mitchell, and S. Connelly. *Zero Trust Architecture*. National Institute of Standards and Technology, Aug. 2020.

[24] S. Talebi. auth.py, Oct. 2025.

[25] S. Talebi. How to build a remote mcp server (with auth), Nov. 2025.

[26] Wikipedia. Model context protocol, Mar. 2026. Page Version ID: 1341517807.